

Assignment 1 · Theory

1) What is tokenization? Give an example — show how the sentence "I'm learning NLP in 2025!" would be tokenized.

Tokenization is the process of splitting text into smaller units called tokens, so that computers and machine learning models can process natural language. Depending on the tokenizer, the tokens may represent words, subwords, or individual characters.

Example:

Sentence:

"I'm learning NLP in 2025!"

One possible tokenization is:

["I'm", "learning", "NLP", "in", "2025", "!"]

Using TensorFlow's `Tokenizer`, punctuation is removed by default, so the output would be:

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer

sentence = ["I'm learning NLP in 2025!"]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentence)

word_index = tokenizer.word_index
print(word_index)

# Output:
# {"i'm": 1, "learning": 2, "nlp": 3, "in": 4, "2025": 5}
```

This example shows that the exact tokens depend on the tokenizer being used.

2) What is the difference between stemming and lemmatization? Apply both to the words "running" and "better" and explain which preserves more linguistic meaning.

Stemming and lemmatization are both techniques used in natural language processing to reduce words to their base form.

- - **Stemming** removes prefixes or suffixes from words to reduce them to their root or stem form, without considering the word's actual meaning.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["running", "better"]
stemmed_words = [stemmer.stem(word) for word in words]

print(stemmed_words)

# Output:
# ['run', 'better']
```

- - **Lemmatization** uses linguistic rules and vocabulary to convert words to their dictionary form (lemma), preserving their linguistic meaning.

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

lemmas = [
    lemmatizer.lemmatize("running", pos=wordnet.VERB),
    lemmatizer.lemmatize("better", pos=wordnet.ADJ),
]

print(lemmas)

# Output:
# ['run', 'good']
```

Lemmatization preserves more linguistic meaning than stemming because it considers the word's grammatical role and returns its correct dictionary form rather than simply removing prefixes or suffixes.

3) What does TF-IDF stand for? Explain in plain language why the word "the" scores almost zero in TF-IDF, while the word "photosynthesis" would score high.

It is an acronym for **Term Frequency–Inverse Document Frequency**.

TF-IDF is a statistical method used to measure how important a word is to a document within a collection of documents (a corpus).

- - **Term Frequency (TF)** measures how many times a word appears in a specific document.
- - **Inverse Document Frequency (IDF)** measures how rare a word is across the collection of documents by considering how many documents contain the word.

The TF-IDF score is calculated by multiplying TF and IDF. A high score means the word appears frequently in the current document but rarely in other documents, making it important. A low score means the word appears in many documents, so it carries little useful information.

For example, the word "the" appears in almost every document, so its IDF is very low and its TF-IDF score is close to zero. In contrast, the word "photosynthesis" appears in relatively few documents but is highly relevant when it does appear, so it receives a much higher TF-IDF score.

4) What is a sentence embedding? How is it fundamentally different from one-hot encoding? Give one advantage embeddings have that one-hot vectors don't.

A sentence embedding is a method used to represent a sentence as a vector based on its semantic meaning. Sentences with similar meanings are represented by vectors that are close to each other in the embedding space, while sentences with different meanings are represented by vectors that are farther apart.

In contrast, one-hot encoding gives each unique word its own binary vector. The length of the vector equals the size of the vocabulary, and the representation does not capture any semantic meaning or relationship between words.

One advantage of sentence embeddings is that they capture semantic similarity, making it possible to perform semantic search, find similar sentences, and cluster texts based on their meaning, whereas one-hot vectors cannot.

5) Explain cosine similarity in plain language. If two document vectors point in almost the same direction, what does that tell us about the documents they represent?

Cosine similarity is a method used to measure how similar two embedding vectors are by comparing the angle between them. The smaller the angle between the vectors, the more similar they are.

If two document vectors point in almost the same direction, it means that the documents have very similar semantic meaning. The closer the vectors point to the same direction, the more similar the documents are.

****Bonus:**** Euclidean distance can be a poor choice for comparing embeddings because it is heavily affected by vector magnitude. Two embeddings may have similar semantic meaning but different magnitudes, making their Euclidean distance large even though they are semantically similar.

6) Why can't a regular SQL query like `WHERE description LIKE '%pizza%'` find semantically similar documents? What does a vector index solve that SQL can't?

SQL is a query language used to store, manage, and retrieve data from relational databases. A query such as `WHERE description LIKE '%pizza%'` searches for exact text matches, not semantic meaning. It can only find documents that contain the word **pizza**, but it cannot find documents with similar meaning that use different words or phrases.

A vector index solves this problem by searching for the nearest embedding vectors based on semantic similarity. It can retrieve semantically related words such as **dough**, **pasta**, or **pepperoni**, related sentences such as **"I love Italian food,"** and even entire documents relevant to pizza even if they do not contain the exact word **pizza**.

7) What problem does RAG solve that a plain LLM (without RAG) cannot? Give a concrete example of when you would choose RAG over just prompting the LLM directly. Think about: knowledge cutoffs, private data, hallucinations, up-to-date information.

RAG gives an LLM access to relevant external information instead of relying only on the knowledge it learned during training. When a user asks a question, the retrieval system searches the provided documents or database for the most relevant information and supplies it to the LLM before it generates an answer.

RAG helps solve several problems:

- - LLMs have a knowledge cutoff and may not know recent information, while RAG can retrieve up-to-date information.
- - LLMs do not have access to private company data, but RAG can retrieve information from private databases or documents without retraining the model.
- - RAG reduces hallucinations because the LLM generates its answer using relevant retrieved information instead of relying only on its internal knowledge.

For example, if I want an AI assistant for my online clothing store, a regular LLM would not have access to my inventory database, product catalog, or company information. With RAG, the assistant can retrieve the relevant inventory and product data and answer customer questions, recommend available clothing items, and provide accurate information based on the current database.

8) Describe the 3 main steps of a RAG pipeline in the correct order. Be clear about what happens at ingestion time (when you load documents) vs query time (when a user asks a question).

The 3 steps are: chunk → embed → store (ingestion) and embed query → retrieve → generate (query time).

The 3 main stages of a RAG pipeline are: **Ingestion → Retrieval → Generation.**

i) Ingestion

- - **Chunking** – Breaking the information into smaller chunks. There are many strategies for chunking, with the goal of splitting the information into smaller pieces without losing the context and meaning of the original text.
- - **Embedding** – Converting each chunk into an embedding vector using an embedding model.
- - **Storing** – Storing the embedding vectors in a vector database (vector index) so they can be searched later.

ii) Retrieval (Query Time)

The retrieval stage begins when the user asks a question. The user's query is converted into an embedding using the same embedding model used for the document chunks. The query embedding is then used to search the vector index and retrieve the most relevant document chunks using semantic search.

iii) Generation (Query Time)

- - Building a prompt by combining the user's query with the retrieved document chunks.
- - Sending the prompt to the LLM, which generates an answer based on both the retrieved information and its own reasoning.

9) What is the difference between a Docker image and a Docker container? Use an analogy to explain.

- - A Docker image is like a recipe that contains everything needed to run a program in Docker, including the application code, libraries, dependencies, runtime, configuration, and instructions.
- - A Docker container is the running instance created from that image. It is like the meal prepared using the recipe. You can create multiple containers from the same image, just like you can cook the same recipe multiple times.

10) What is the difference between a simple LLM chatbot and an AI agent with tools? Give one concrete example of a "tool" and explain why it makes the agent more capable.

An LLM chatbot does not have any tools to act outside of the chat or interact with the outside world.

An AI agent can use tools for booking reservations, sending emails, searching the internet, running SQL queries, writing code, and automating tasks. For instance, if I wanted help managing my time, an LLM could only suggest meeting times and schedules, while an AI agent could actually create events in my Google Calendar and manage my schedule instead of just making suggestions.

11) What is MCP (Model Context Protocol)? What problem does it solve for AI coding assistants like GitHub Copilot? Name two examples of things an MCP server might expose to an AI assistant.

MCP is a protocol that allows LLMs to connect to external tools and systems through a standard interface. Without MCP, an AI coding assistant such as GitHub Copilot would need a different API or custom integration for every external tool, making AI systems difficult to build, maintain, and scale. MCP solves this problem by providing one standard protocol that allows the AI assistant to communicate with many different tools in the same way.

For example, if you want to use GitHub Copilot in your project, without MCP you would need a different integration for every tool your project uses. With MCP, the assistant can communicate with many different tools through the same protocol, making the system much easier to build and scale.

Examples of tools or resources an MCP server might expose include:

- - **File System** – allowing the AI assistant to read, create, edit, and search project files.
- - **Terminal** – allowing the AI assistant to execute shell commands such as running tests, installing packages, or building the project.
- - **Git Repository** – allowing the AI assistant to inspect commits, branches, and create pull requests.
- - **Database** – allowing the AI assistant to query or update information stored in a database.
- - **Documentation** – allowing the AI assistant to access internal documentation or API references.
- - **Browser** – allowing the AI assistant to browse websites or interact with web applications.

12) What are Agent Skills in the context of AI coding assistants? How are they different from just writing instructions in a plain prompt? Show a minimal example of what a skill's `.md` metadata block might look like.

Agent Skills are different from simply writing instructions in a prompt. They allow the AI agent to reuse a predefined workflow (procedural instructions) for a specific task.

For example, suppose you want your agent to generate a legal document. Without Agent Skills, you have two options:

- - Write a long prompt describing every step required to create the document.
- - Let the LLM guess how to perform the task.

By adding a skill `.md` file, you provide the agent with a reusable workflow that defines how to perform a specific task, which tools to use, and in what order to use them.

A minimal skill metadata block contains:

- - `name` – the name of the skill (e.g., "PDF Builder")
- - `description` – when the skill should be used (e.g., "Use this skill when the user asks to create or extract a PDF.")
- - `body` – the actual instructions or workflow the agent should follow.

A skill can also include optional folders such as:

- - `scripts/` – code that the agent can execute.
- - `references/` – additional documentation relevant to the skill.
- - `assets/` – static resources such as templates, images, or data files.

Example:

```
<skill>
  <name>code-reviewer</name>
  <description>
    Help review Python code.
    Use when the user asks "Can you review my code?"
    or "Can you find bugs in this Python program?"
  </description>
  <file>path/to/skills/code-reviewer/SKILL.md</file>
</skill>
```